

Проект по предметот Бази на Податоци

Дизајн и имплементација на систем за управување со резервации и деловни договори во заеднички работни центри (Coworking Spaces) - CoWorkHub

Мартина Петковска, 223313

Факултет за информатички науки и компјутерско инженерство, Скопје
Ментор: д-р Горан Велинов

Апстракт: Овој проект го презентира CoWorkHub, систем кој овозможува флексибилно изнајмување на работен простор преку поддршка на еднократни резервации и долгорочни деловни договори. Системот користи напреден модел на податоци за динамичко конфигурирање на просторот и автоматизирано генерирање на фактури.

Клучни зборови: Coworking, Резервации, Деловни договори, Фактури.

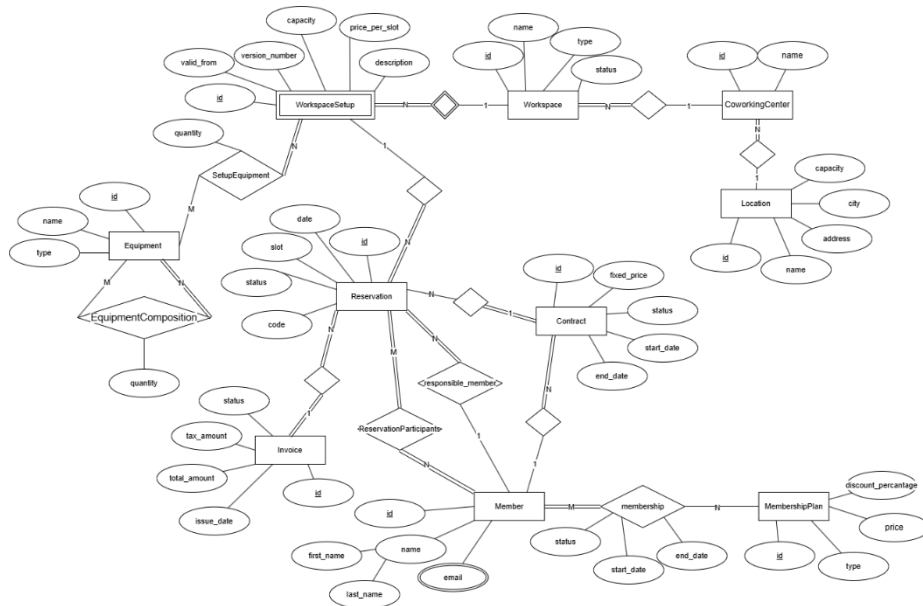
1 Опис на функционалностите на системот

CoWorkHub претставува систем за управување со резервации и деловни договори во Coworking центри. Системот поддржува два модели на користење: еднократни резервации за индивидуални корисници и долгорочен закуп преку деловни договори.

Системот овозможува управување со центри, нивните локации и работни простори. За секој центар се чува неговото име, додека локациите содржат детални податоци за адресата. Секој центар располага со повеќе работни простори (Workspace), кои имаат дефиниран тип, капацитет и статус. Работните простори можат да имаат различни конфигурации (WorkspaceSetup) низ времето, кои содржат специфична опрема и нејзини под-компоненти.

Членовите можат да користат различни временски претплати (MembershipPlan) и да вршат еднократни резервации (Reservation) за конкретен сетоп или закуп преку договори (Contract). Во рамки на резервација, системот автоматски генерира фактура (Invoice). Додека, во рамките на договорите, системот поддржува закуп на повеќе работни единици со дефиниран попуст и времетраење, како и автоматско генерирање месечни фактури (Invoice) за сите активности поврзани со договорот.

2 Опис на моделот на податоци



2.1 Основни ентитети

1. **Работни Простори:** CoworkingCenter, Location, Workspace, WorkspaceSetup, Equipment
2. **Членство:** Member, MembershipPlan
3. **Трансакции:** Reservation, Contract, Invoice

2.2 Релационен модел на податоци

Релациониот модел е дефиниран преку следните релации:

Location(id, name, address, city)

CoworkingCenter(id, name, location_id*)

Workspace(id, name, capacity, type, status, coworking_center_id*)

WorkspaceSetup(id, workspace_id*, version_number, valid_from, price_per_slot, description)

Equipment(id, name, type)

SetupEquipment(setup_id*, equipment_id*, quantity)

EquipmentComposition(parent_equipment_id*, child_equipment_id*, quantity)

Member(id, first_name, last_name)

MemberEmail(member_id*, email)

MembershipPlan(id, type, price, discount_percentage)

Membership(member_id*, plan_id*, start_date, end_date, status)

Reservation(id, date, slot, status, code, responsible_member_id*, setup_id*, invoice_id*, contract_id*)

ReservationParticipants(reservation_id*, member_id*)

Invoice(id, issue_date, total_amount, tax_amount, status)

Contract(id, fixed_price, status, start_date, end_date, member_id*)

2.3 Опис на моделот на податоци

Моделот на податоци на системот CoWorkHub е организиран во неколку логички целини: управување со локации и центри, работни простори, членство и трансакции. Ентитетите Location и CoworkingCenter овозможуваат евиденција на физички локации и центри, додека Workspace и WorkspaceSetup поддржуваат различни конфигурации на работни простори со специфична цена, капацитет и намена.

За управување со опрема се користат Equipment, SetupEquipment и EquipmentComposition, со што се овозможува поврзување на повеќе типови опрема и нивни составни делови. Корисниците се моделирани преку Member, MemberEmail, MembershipPlan и Membership, што овозможува поддршка за повеќе претплатнички планови.

Трансакцискиот дел е имплементиран преку Reservation, ReservationParticipants, Invoice и Contract, при што системот поддржува еднократни резервации и долгорочни деловни договори со автоматско фактурирање.

3 Пристап до податоците

3.1 SQL имплементација

3.1.1 Креирање на табели (DDL)

Системот CoWorkHub е имплементиран во PostgreSQL и користи релационен модел со 15 меѓусебно поврзани табели. Во базата се дефинирани примарни и надворешни клучеви, CHECK ограничувања, UNIQUE правила и индекси за оптимизација на пребарувања по датум и корисник.

Приказ на дел од структурата:

```
CREATE TABLE Location ( id SERIAL ... );
CREATE TABLE Equipment ( id SERIAL ... );
CREATE TABLE Member ( id SERIAL ... );
CREATE TABLE MembershipPlan ( id SERIAL ... );
CREATE TABLE Invoice ( id SERIAL ... );
CREATE TABLE CoworkingCenter ( id SERIAL ... );
CREATE TABLE Workspace ( id SERIAL ... );
CREATE TABLE WorkspaceSetup ( id SERIAL ... );
CREATE TABLE Contract ( id SERIAL ... );
CREATE TABLE Reservation ( id SERIAL ... );
CREATE TABLE SetupEquipment ( setup_id ... );
CREATE TABLE EquipmentComposition ( parent_equipment_id ... );
CREATE TABLE ReservationParticipants ( reservation_id ... );
CREATE TABLE Membership ( id SERIAL ... );
CREATE TABLE MemberEmail ( member_id ... );
```

```
CREATE TABLE Reservation (
  id SERIAL PRIMARY KEY,
  date DATE NOT NULL,
  slot VARCHAR(20) NOT NULL
  CHECK (slot IN ('morning', 'afternoon', 'evening')),
  status VARCHAR(20) NOT NULL DEFAULT 'pending'
  CHECK (status IN ('pending', 'confirmed', 'cancelled')),
  code VARCHAR(20) NOT NULL UNIQUE,
  responsible_member_id INT NOT NULL,
  setup_id INT NOT NULL,
  invoice_id INT NULL,
  contract_id INT NULL,
  FOREIGN KEY (responsible_member_id) REFERENCES Member(id)
  ON DELETE RESTRICT ON UPDATE CASCADE,
  FOREIGN KEY (setup_id) REFERENCES WorkspaceSetup(id)
  ON DELETE RESTRICT ON UPDATE CASCADE,
  FOREIGN KEY (invoice_id) REFERENCES Invoice(id)
  ON DELETE RESTRICT ON UPDATE CASCADE,
  FOREIGN KEY (contract_id) REFERENCES Contract(id)
  ON DELETE RESTRICT ON UPDATE CASCADE,
  UNIQUE (setup_id, date, slot)
);
```

```
CREATE TABLE Invoice(
  id SERIAL PRIMARY KEY,
  issue_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  total_amount NUMERIC(10,2) NOT NULL
  CHECK (total_amount >= 0),
  tax_amount NUMERIC(10,2) NOT NULL,
  status VARCHAR(20) DEFAULT 'unpaid'
  CHECK (status IN ('paid', 'unpaid', 'refunded')),
  type VARCHAR(50)
  CHECK (type IN ('single', 'contract'))
);
```

Додадени се индекси за оптимизација:

```
-- index for faster queries
CREATE INDEX idx_reservation_date ON Reservation (date);
CREATE INDEX idx_reservation_member ON Reservation(responsible_member_id);
CREATE INDEX idx_membership_member ON Membership(member_id);
```

3.1.2 Процедури

Во системот се имплементирани складирани процедури (Stored Procedures) за автоматизација на дел од бизнис логиката поврзана со фактурирање и обработка на резервации. Процедурите се повикуваат од Django апликацијата преку signals при креирање и ажурирање на резервации.

Имплементирани се следниве процедури:

1. **generate_single_invoice** – генерира индивидуална фактура за резервација без договор

```
CREATE OR REPLACE PROCEDURE
generate_single_invoice(p_reservation_id INT,
p_setup_id INT, p_member_id INT)
LANGUAGE plpgsql AS $$
DECLARE
    base_price NUMERIC(10,2);
    discount DECIMAL(5,2) DEFAULT 0;
    final_price NUMERIC(10,2);
    new_invoice_id INT;
BEGIN
    SELECT price_per_slot INTO base_price
    FROM WorkspaceSetup WHERE id = p_setup_id;

    SELECT COALESCE(mp.discount_percentage, 0) INTO
discount
    FROM Membership m
    LEFT JOIN MembershipPlan mp ON m.plan_id =
mp.id
    WHERE m.member_id = p_member_id
    AND m.status = 'active'
    AND CURRENT_DATE BETWEEN m.start_date AND
m.end_date
    LIMIT 1;

    final_price := base_price * (1 - discount / 100);

    INSERT INTO Invoice (total_amount, tax_amount,
status, type)
VALUES (final_price, final_price * 0.18, 'unpaid',
'single')
RETURNING id INTO new_invoice_id;

    UPDATE Reservation
    SET invoice_id = new_invoice_id
    WHERE id = p_reservation_id;
END;
$$;
```

2. **generate_contract_invoice** – групира договорни резервации и генерира заедничка фактура

```
CREATE OR REPLACE PROCEDURE
generate_contract_invoice(p_reservation_id INT,
p_setup_id INT, p_contract_id INT)
LANGUAGE plpgsql AS $$
DECLARE
    slot_price NUMERIC(10,2);
    existing_invoice_id INT;
BEGIN
    SELECT price_per_slot INTO slot_price
    FROM WorkspaceSetup WHERE id = p_setup_id;

    SELECT invoice_id INTO existing_invoice_id
    FROM Reservation
    WHERE contract_id = p_contract_id
        AND invoice_id IS NOT NULL
        AND id != p_reservation_id
    LIMIT 1;

    IF existing_invoice_id IS NOT NULL THEN
        UPDATE Invoice
        SET total_amount = total_amount + slot_price,
            tax_amount = tax_amount + (slot_price * 0.18)
        WHERE id = existing_invoice_id;

        UPDATE Reservation
        SET invoice_id = existing_invoice_id
        WHERE id = p_reservation_id;
    ELSE
        INSERT INTO Invoice (total_amount, tax_amount,
status, type)
VALUES (slot_price, slot_price * 0.18, 'unpaid',
'contract')
RETURNING id INTO existing_invoice_id;

        UPDATE Reservation
        SET invoice_id = existing_invoice_id
        WHERE id = p_reservation_id;
    END IF;
END;
$$;
```

3. **cancel_reservation_invoice** – ажурира фактура при откажување на резервација

```
CREATE OR REPLACE PROCEDURE cancel_reservation_invoice(
    p_reservation_id INT,
    p_invoice_id INT,
    p_contract_id INT
)
    LANGUAGE plpgsql AS $$
DECLARE
    slot_price NUMERIC(10,2);
    v_setup_id INT;
BEGIN
    IF p_invoice_id IS NULL THEN
        RETURN;
    END IF;

    IF p_contract_id IS NULL THEN
        UPDATE Invoice
        SET total_amount = 0,
            tax_amount = 0,
            status = 'refunded'
        WHERE id = p_invoice_id;
    ELSE
        SELECT setup_id INTO v_setup_id
        FROM Reservation WHERE id = p_reservation_id;

        SELECT price_per_slot INTO slot_price
        FROM WorkspaceSetup WHERE id = v_setup_id;

        UPDATE Invoice
        SET total_amount = GREATEST(total_amount -
slot_price, 0),
            tax_amount = GREATEST(tax_amount -
(slot_price * 0.18), 0)
        WHERE id = p_invoice_id;
    END IF;
END;
$;
```

Во Django апликацијата се имплементирани **сигнали** за автоматско повикување на PostgreSQL процедурите при креирање и ажурирање на резервации. На овој начин се овозможува синхронизација помеѓу апликацискиот слој и бизнис логиката имплементирана во базата.

Главната цел е автоматско генерирање на фактура при секоја нова резервација без дополнителна интервенција од корисникот. Кај резервации поврзани со активен договор, новите резервации автоматски се групираат во постоечка фактура, при што вкупниот износ се ажурира согласно новата резервација.

```

@receiver(post_save, sender=Reservation)
def handle_reservation_save(sender, instance, created,
**kwargs):
    if created:
        with connection.cursor() as cursor:
            if instance.contract_id is None:
                cursor.execute(
                    "CALL generate_single_invoice(%s, %s,
%s)",
                    [instance.id, instance.setup_id,
instance.responsible_member_id]
                )
            else:
                cursor.execute(
                    "CALL generate_contract_invoice(%s,
%s, %s)",
                    [instance.id, instance.setup_id,
instance.contract_id]
                )

@receiver(pre_save, sender=Reservation)
def handle_reservation_cancel(sender, instance,
**kwargs):
    if not instance.pk:
        return
    try:
        old = Reservation.objects.get(pk=instance.pk)
        if old.status != 'cancelled' and instance.status
== 'cancelled':
            with connection.cursor() as cursor:
                cursor.execute(
                    "CALL cancel_reservation_invoice(%s,
%s, %s)",
                    [instance.id, instance.invoice_id,
instance.contract_id]
                )
    except Reservation.DoesNotExist:
        pass

```

3.1.3 Иницијални тест податоци – seed_data.sql

За потребите на тестирање и демонстрација, во системот е внесен иницијален сет на тест податоци (seed data):


```

INSERT INTO Location (name, address, city) VALUES ('Centar'...;

INSERT INTO Equipment (name, type) VALUES ('Dell Monitor 27"...'...;

INSERT INTO Member (first_name, last_name) VALUES ('Petar'...;

INSERT INTO MembershipPlan (type, price, discount_percentage) VALUES ('Basic'...;

INSERT INTO CoworkingCenter (name, contact_phone, email, location_id) VALUES ('Hub Skopje Centar'...;

INSERT INTO Workspace (name, type, capacity, status, coworking_center_id) VALUES ('Room 101'...;

INSERT INTO WorkspaceSetup (workspace_id, version_number, price_per_slot, description) VALUES (1,...;

INSERT INTO Contract (fixed_price, status, start_date, end_date, member_id) VALUES
  (fixed_price 25000.00, status 'active', start_date '2026-01-01', end_date '2026-06-30', member_id 1),
  (fixed_price 18000.00, status 'active', start_date '2026-02-01', end_date '2026-07-31', member_id 2),
  (fixed_price 10000.00, status 'terminated', start_date '2025-09-01', end_date '2025-12-31', member_id 3),
  (fixed_price 45000.00, status 'active', start_date '2026-04-01', end_date '2026-09-30', member_id 4),
  (fixed_price 15000.00, status 'active', start_date '2026-03-01', end_date '2026-08-31', member_id 3);

INSERT INTO Reservation (date, slot, status, code, responsible_member_id, setup_id, invoice_id, contract_id) VALUES
  (date '2026-05-25', slot 'morning', status 'confirmed', code 'RES-SKP-015', responsible_member_id 1, setup_id 1, invoice_id NULL, contract_id NULL),
  (date '2026-05-27', slot 'afternoon', status 'pending', code 'RES-SKP-020', responsible_member_id 5, setup_id 1, invoice_id NULL, contract_id NULL),
  (date '2026-06-01', slot 'morning', status 'confirmed', code 'RES-SKP-025', responsible_member_id 1, setup_id 1, invoice_id NULL, contract_id NULL),
  (date '2026-06-01', slot 'afternoon', status 'confirmed', code 'RES-SKP-026', responsible_member_id 2, setup_id 1, invoice_id NULL, contract_id NULL),
  (date '2026-06-02', slot 'morning', status 'confirmed', code 'RES-SKP-027', responsible_member_id 3, setup_id 1, invoice_id NULL, contract_id NULL);

```

3.2 Извештај 1 - Финансиски преглед на фактури

Цел: За потребите на финансиска анализа е креиран поглед (VIEW) кој овозможува месечен преглед на фактури според тип и статус. Извештајот прикажува вкупни платени, неплатени и рефундирани износи, број на фактури по статус, како и вкупен данок и генериран приход.

```

CREATE VIEW view_invoice_ledger AS
SELECT
  DATE_TRUNC('month', i.issue_date)::DATE AS month,
  i.type AS invoice_type,
  SUM(CASE WHEN i.status = 'paid' THEN
i.total_amount ELSE 0 END) AS total_paid,
  SUM(CASE WHEN i.status = 'unpaid' THEN
i.total_amount ELSE 0 END) AS total_unpaid,
  SUM(CASE WHEN i.status = 'refunded' THEN
i.total_amount ELSE 0 END) AS total_refunded,
  COUNT(CASE WHEN i.status = 'paid' THEN 1 END) AS
count_paid,
  COUNT(CASE WHEN i.status = 'unpaid' THEN 1 END) AS
count_unpaid,
  COUNT(CASE WHEN i.status = 'refunded' THEN 1 END) AS
count_refunded,
  SUM(i.total_amount) AS total_amount,
  SUM(i.tax_amount) AS total_tax
FROM Invoice i
GROUP BY DATE_TRUNC('month', i.issue_date), i.type
ORDER BY month DESC, i.type;

SELECT * FROM view_invoice_ledger;

```

3.3 Извештај 2 – Пополнетост на центри

Цел: Овој извештај овозможува прецизен мониторинг на искористеноста на физичкиот простор преку споредба на вкупниот капацитет на центрите со реално направените резервации на специфичен датум.

```
CREATE VIEW center_occupancy AS
SELECT
    cc.id AS center_id,
    cc.name AS center_name,
    l.city,
    r.date AS reservation_date,
    COUNT(r.id) AS total_bookings,
    SUM(w.capacity) AS total_capacity_used,
    (SELECT SUM(capacity) FROM Workspace WHERE
    coworking_center_id = cc.id) AS total_capacity
FROM CoworkingCenter cc
    JOIN Location l ON cc.location_id = l.id
    JOIN Workspace w ON w.coworking_center_id =
cc.id
    JOIN WorkspaceSetup ws ON ws.workspace_id = w.id
    JOIN Reservation r ON r.setup_id = ws.id
WHERE r.status = 'confirmed'
GROUP BY cc.id, l.city, r.date
ORDER BY r.date DESC, total_bookings DESC;

SELECT * FROM center_occupancy
WHERE reservation_date = CURRENT_DATE;
```

4 Опис на апликацијата

CoWorkHub е database-first апликација имплементирана со PostgreSQL и Django, наменета за управување со coworking центри, резервации, договори и фактурирање. Бизнис логиката е имплементирана во базата преку складирани процедури, додека Django signals овозможуваат нивно автоматско повикување при работа со резервации, како и административен интерфејс.

4.1 Административен панел

Апликацијата користи Django Admin како главен интерфејс за управување со податоците. Преку административниот панел се овозможува CRUD функционалност за членови, претплати, резервации, договори, работни простори и конфигурации.

На сликата подолу е прикажана табелата за Резервации:

CoworkHub

REPORTSADMINLOGOUT

Home · Coworkhub · Reservations

Start typing to filter...

Groups + Add

Users + Add

AUTHENTICATION AND AUTHORIZATION

Contracts + Add

Coworking centers + Add

Equipments + Add

Invoices + Add

Locations + Add

Members + Add

Membership plans + Add

Reservations + Add

Workspace setups + Add

Workspaces + Add

COWORKHUB

Select reservation to change

ADD RESERVATION +

Q Search

All dates May 2026 June 2026

Actions: 0 of 12 selected

☐	ID	CODE	DATE	SLOT	STATUS	RESPONSIBLE MEMBER	WORKSPACE	INVOICE	CONTRACT
☐	16	RES-2A06DA99	May 15, 2026	Morning	Pending	Ana Angelovska	Open Desk A	Invoice #12	-
☐	11	aaa	May 12, 2026	Evening	Pending	Mihail Georgiev	Conference Room Alpha	Invoice #9	-
☐	10	RES-SKP-030	May 12, 2026	Morning	Cancelled	Ana Angelovska	Open Desk A	Invoice #6	Contract object (2)
☐	9	RES-SKP-029	June 3, 2026	Morning	Confirmed	Martina Petkovska	Conference Room Alpha	Invoice #6	Contract object (2)
☐	8	RES-SKP-028	June 2, 2026	Afternoon	Confirmed	Mihail Georgiev	Room 101	Invoice #7	Contract object (4)
☐	7	RES-SKP-027	June 2, 2026	Morning	Confirmed	Petar Petrovski	Room 101	Invoice #5	Contract object (1)
☐	6	RES-SKP-026	June 1, 2026	Afternoon	Confirmed	Martina Petkovska	Room 101	Invoice #6	Contract object (2)
☐	5	RES-SKP-025	June 1, 2026	Morning	Confirmed	Petar Petrovski	Room 101	Invoice #5	Contract object (1)
☐	4	RES-SKP-022	May 28, 2026	Afternoon	Cancelled	Mihail Georgiev	Conference Room Alpha	Invoice #4	-
☐	3	RES-SKP-021	May 28, 2026	Morning	Confirmed	Ana Angelovska	Conference Room Alpha	Invoice #3	-
☐	2	RES-SKP-020	May 27, 2026	Afternoon	Pending	Stefan Trajkovski	Open Desk A	Invoice #2	-
☐	1	RES-SKP-015	May 25, 2026	Morning	Cancelled	Petar Petrovski	Open Desk A	Invoice #1	-

12 reservations

4.2 Форма за внес на резервации

Следниот екран ја покажува формата за внес на резервација, каде корисникот внесува датум, термин, workspace setup, одговорен член и дали резервацијата е дел од деловен договор или не. По зачувување на податоците, автоматски се генерира или ажурира соодветна фактура.

CoworkHub

New ReservationReportsAdmin

New Reservation

Member

Workspace Setup

Date

05/12/2026

Slot

Morning

Contract (optional)

Leave empty for a single reservation with automatic invoice.

Create Reservation

4.3 Извештаи

На екранот подолу е прикажана посебна страница за приказ на аналитички податоци од системот. Преку SQL views се прикажуваат финансиски извештај за фактури и извештај за пополнетост на coworking центрите.

CoworkHub

ReportsAdmin

Reports

Invoice Status Ledger

Month	Invoice Type	Total Paid	Total Unpaid	Total Refunded	Count Paid	Count Unpaid	Count Refunded	Total Amount	Total Tax
May 1, 2026	contract	500.00	2700.00	0	1	2	0	3200.00	576.00
May 1, 2026	single	780.00	1275.00	0.00	1	2	1	2055.00	369.90

Center Occupancy

Center Id	Center Name	City	Reservation Date	Total Bookings	Total Capacity Used	Total Capacity
2	Innovation Lab Karposh	Skopje	June 3, 2026	1	12	12
1	Hub Skopje Centar	Skopje	June 2, 2026	2	8	5
1	Hub Skopje Centar	Skopje	June 1, 2026	2	8	5
2	Innovation Lab Karposh	Skopje	May 28, 2026	2	24	12

5 Заклучок

CoWorkHub претставува интегриран систем за управување со coworking простори, резервации и деловни договори. Со релационен модел, напредни SQL ограничувања и автоматизирани процедури и Django сигнали, системот обезбедува ефикасно управување со ресурси и финансии. Имплементацијата овозможува конзистентност на податоци и реална симулација на современи coworking бизнис процеси.

References

1. <https://www.postgresql.org/docs/current/>
2. <https://www.postgresql.org/docs/current/functions.html>
3. <https://www.psycopg.org/install/>
4. <https://docs.djangoproject.com/en/6.0/ref/databases/#postgresql-notes>